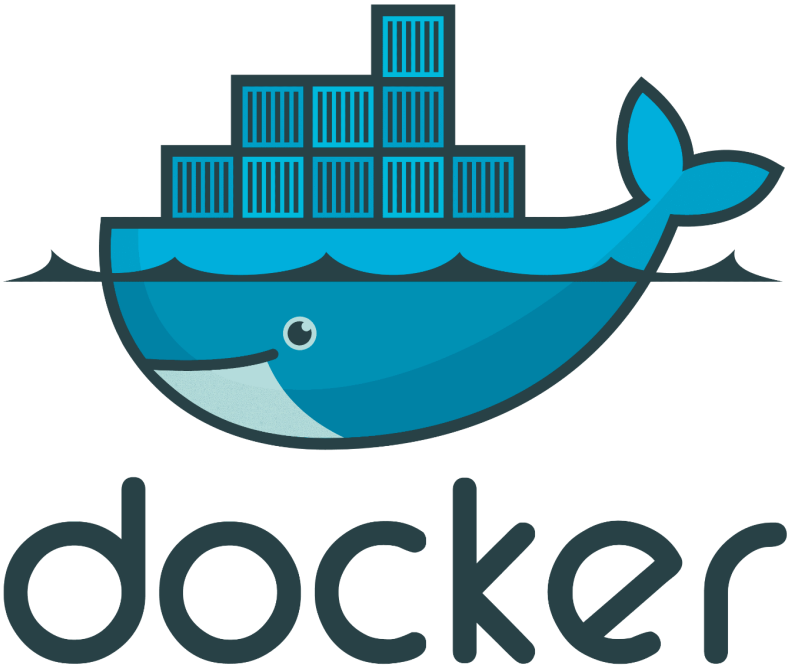
DICCIONARIO DE DOCKER (2)



DOCKER: LA ENCICLOPEDIA TOTAL

De la ingeniería del Kernel a la orquestación de élite.

Consta de: Un desglose técnico absoluto en 8 tomos que cubre desde la creación de imágenes optimizadas y seguridad avanzada (Hardening), hasta el despliegue de microservicios, IA y clústeres a escala global con Kubernetes.

Dirigido a: Desarrolladores, Arquitectos y DevOps que no buscan un tutorial rápido, sino dominar la infraestructura de contenedores al nivel de los expertos que diseñan los sistemas de las grandes tecnológicas.

📑 ÍNDICE GENERAL (Estructura de Élite)

TOMO I: EL MOTOR Y LA ARQUITECTURA OCULTA

- **Capítulo 1:** Anatomía Forense del Contenedor (Namespaces, Cgroups y Capabilities).
- **Capítulo 2:** El Ecosistema OCI: dockerd, containerd y runc.

TOMO II: GESTIÓN DE IMÁGENES Y CONSTRUCCIÓN DE ÉLITE

- **Capítulo 3:** Ciclo de Vida y Anatomía de Capas (UnionFS & CoW).
- **Capítulo 4:** Dockerfiles de Alto Rendimiento: Multi-stage builds y Distroless.

TOMO III: OPERACIONES, REDES Y PERSISTENCIA

- **Capítulo 5:** Redes en Docker: Del Bridge a las redes Overlay.
- **Capítulo 6:** El Triángulo de la Persistencia: Volumes, Bind Mounts y Tmpfs.

TOMO IV: ORQUESTACIÓN CON DOCKER SWARM

- **Capítulo 7:** Swarm Mode: Servicios, Stacks y Healthchecks.
- **Capítulo 8:** Gestión de Secretos y Configuración en Producción.

TOMO V: KUBERNETES – EL ESTÁNDAR DE ORQUESTACIÓN

- **Capítulo 9:** Arquitectura de K8s: El Plano de Control y los Nodos.
- **Capítulo 10:** Objetos de K8s: Pods, Deployments y Services.

TOMO VI: SEGURIDAD Y HARDENING (NIVEL DIOS)

- **Capítulo 11:** Escaneo de Vulnerabilidades y Firmado de Imágenes.
- **Capítulo 12:** Hardening del Host y Perfiles Seccomp/AppArmor.

TOMO I: EL MOTOR Y LA ARQUITECTURA OCULTA

CAPÍTULO 1: Anatomía Forense del Contenedor

1. Abstracción de Alto Nivel (Concepto)

Un contenedor es un **proceso aislado** que comparte el Kernel del sistema operativo anfitrión. A diferencia de las Máquinas Virtuales (VM), que requieren un Hypervisor para emular hardware y ejecutar un Kernel completo (Guest OS), el contenedor utiliza las funciones nativas de Linux para crear un entorno estanco. Esto elimina la sobrecarga de memoria y CPU asociada a la virtualización tradicional.

- **Diferencia clave:** En una VM, si tienes 10 servicios, tienes 10 Kernels cargados en RAM. En Docker, tienes 10 procesos aislados hablando con el mismo Kernel, lo que permite arranques en milisegundos.

2. Arquitectura de Bajo Nivel (Deep Dive al Kernel)

Para que un proceso crea que es una máquina independiente, Docker orquestra tres tecnologías del Kernel:

- **Namespaces (Aislamiento de visión):** * **mnt (Mount):** Proporciona al proceso su propio árbol de directorios. No puede ver el / del host.
    - **pid (Process ID):** El proceso principal se ve a sí mismo como PID 1. Esto es vital porque en Linux, el PID 1 es el encargado de gestionar señales y procesos hijos.
    - **net (Network):** El proceso recibe su propio stack de red, tablas de rutas e IP.
    - **uts:** Permite tener un *hostname* y nombre de dominio propio.
    - **ipc:** Aísla la comunicación entre procesos (memoria compartida).
- **Control Groups / Cgroups (Gestión de Recursos):**
  Es el subsistema que limita cuánto puede consumir el proceso. Sin Cgroups, un contenedor con un bucle infinito de memoria podría provocar un **Kernel Panic** o activar el **OOM Killer** (Out Of Memory Killer), cerrando procesos críticos del servidor.
- **Capabilities (Privilegios Segmentados):**
  Históricamente, los procesos eran root o no lo eran. Docker utiliza las *capabilities* de Linux para dividir los privilegios del superusuario en 40 categorías. Por defecto, Docker deniega casi todas (como CAP_SYS_REBOOT, impidiendo que un contenedor reinicie el servidor físico).

3. Diccionario de Comandos "Power User"

Comando	Flag Avanzado	Explicación Técnica
docker run	--cpuset-cpus="0,1"	Fuerza al contenedor a ejecutarse solo en los núcleos 0 y 1 de la CPU.
docker run	--pids-limit 100	Previene ataques de agotamiento de recursos (Fork Bombs) limitando los hilos.
docker run	--blkio-weight 500	Prioriza la velocidad de lectura/escritura en disco frente a otros contenedores.
docker inspect	--format '{{.State.Pid}}'	Extrae el ID de proceso del host para depuración externa.

4. Guía de Implementación Paso a Paso (Laboratorio de Aislamiento)

Vamos a demostrar que el contenedor es solo un proceso mediante la "técnica del espejo":

Lanzamiento con límites:
Bash
```
docker run -d --name lab_pro --memory="256m" --cpus="0.5" nginx
```
Localización del PID en el Host:
Bash
```
export PID=$(docker inspect -f '{{.State.Pid}}' lab_pro)
echo "El proceso real en mi servidor es el PID: $PID"
```
Inspección de Namespaces desde el Host:
Usa la herramienta lsns para ver cómo el Kernel ha separado al proceso:
Bash
```
sudo lsns -p $PID
```
(Verás que los IDs de los namespaces de NET, PID y MNT son distintos a los del sistema raíz).

5. El Pipeline de Seguridad (Hardening de Proceso)

Para un nivel máximo de seguridad, debemos aplicar el **Principio de Privilegio Mínimo**:

ReadOnly RootFS: Haz que el sistema de archivos del contenedor sea de solo lectura para evitar que un atacante descargue scripts o modifique binarios:
Bash
docker run --read-only nginx

-

No-New-Privileges: Evita que el contenedor pueda escalar privilegios mediante binarios setuid:
Bash
docker run --security-opt=no-new-privileges nginx

-

6. Optimización de Recursos y Performance (Benchmark)

El driver de almacenamiento overlay2 permite que múltiples contenedores compartan las capas de una imagen.

- **Prueba de rendimiento:** Si ejecutas 50 contenedores de una imagen de 1GB, no gastarás 50GB de disco. Solo gastarás 1GB (la imagen compartida) más unos pocos MB por cada capa de escritura de cada contenedor.
- **Optimización:** Para aplicaciones con alta carga de I/O, utiliza la flag --memory-swappiness=0 para evitar que el Kernel mueva datos de la RAM al disco (Swap), lo cual mataría el rendimiento.

7. Persistencia y Redes (Estado)

El tráfico de red en este nivel inicial se gestiona mediante el **Virtual Ethernet Pair (veth)**:

1. Un extremo del "cable virtual" está dentro del contenedor (eth0).
2. El otro extremo está conectado al bridge docker0 en el host.
3. Docker usa iptables y reglas de **Forwarding** para permitir que el proceso se comunique con el exterior.

8. Troubleshooting Forense (El Método Científico)

- **Escenario:** El contenedor se queda en estado Created pero no arranca.
- **Investigación de Nivel Dios:**
    1. Revisa señales del sistema: docker events en una terminal mientras intentas arrancar en otra.
    2. Verifica el dmesg del host: Si ves un mensaje de "Out of memory", el Cgroup lo está matando antes de nacer.
    3. Usa docker logs aunque el contenedor esté parado para capturar el error de la aplicación.

9. Anti-Patrones (Errores de Producción)

- **Ignorar señales de terminación:** Si tu aplicación no gestiona la señal SIGTERM, Docker la matará violentamente con SIGKILL tras 10 segundos, causando corrupción de datos.
- **Docker-in-Docker (DinD) sin control:** Montar el socket /var/run/docker.sock dentro de un contenedor da acceso total (root) a todo el servidor. Es el agujero de seguridad más común en pipelines de CI/CD.

10. Escalabilidad y Orquestación

A nivel de arquitectura, debemos entender que Docker es **Imperativo** (tú das órdenes: *haz esto*). Sin embargo, el futuro (Kubernetes/Swarm) es **Declarativo** (*quiero que este estado se mantenga*). El Capítulo 1 establece los cimientos: si no entiendes cómo se aísla un proceso en un nodo, no podrás orquestar mil nodos.

CAPÍTULO 2: El Ecosistema OCI: dockerd, containerd y runc

1. Abstracción de Alto Nivel (Concepto)

Docker no es un programa único, sino una **orquesta de componentes especializados**. Lo que instalas como "Docker" es en realidad un conjunto de herramientas que siguen el estándar **OCI (Open Container Initiative)**. Este estándar garantiza que una imagen creada con Docker pueda correr en Kubernetes, Podman o cualquier otro motor compatible.

- **Analogía:** Docker es como el mando a distancia (interfaz). El mando envía una señal al televisor (daemon), pero es el procesador interno del televisor el que realmente genera la imagen en la pantalla.

2. Arquitectura de Bajo Nivel (La Cadena de Mando)

Cuando ejecutas docker run, la orden viaja por una cadena de mando técnica:

1. **Docker CLI:** El cliente que traduce tus comandos a llamadas API REST.
2. **dockerd (The Daemon):** El cerebro. Gestiona imágenes, redes y volúmenes. No crea los contenedores directamente; le da la orden a containerd.
3. **containerd:** Un supervisor industrial de alto rendimiento. Se encarga de descargar las imágenes y gestionar el ciclo de vida (start/stop). Es el mismo motor que usa Kubernetes actualmente (CRI).
4. **runc:** La herramienta de "usar y tirar". Es la que realmente habla con el Kernel de Linux, crea los *Namespaces* y *Cgroups*, lanza el proceso y luego se cierra.
5. **containerd-shim:** El "guardaespaldas". Se queda vivo junto al contenedor para que, si el Daemon de Docker se reinicia, el contenedor no muera.

3. Diccionario de Comandos "Power User"

Comando	Flag / Uso Pro	Propósito en Nivel Máximo
docker version	--format '{{.Server.Os}}'	Verifica detalles exactos del runtime del servidor para asegurar compatibilidad.
docker system info	*(Cualquiera)*	Muestra el driver de almacenamiento y los *Security Options* activos (AppArmor, Seccomp).
ctr	ctr containers list	CLI directa de containerd. Permite gestionar contenedores saltándose a Docker.
docker run	--runtime=[nombre]	Permite cambiar runc por otros ejecutores como **Kata Containers** (VMs ligeras).

4. Guía de Implementación Paso a Paso (Auditando el Motor)

Vamos a observar la cadena de mando en tiempo real:

Lanza un contenedor persistente:
Bash
docker run -d --name motor_test alpine sleep 1000
Rastrea los procesos en el Host:
Ejecuta ps aux | grep -E 'dockerd|containerd|shim'. Verás cómo conviven los tres niveles de la arquitectura.

Prueba de Resiliencia:
Reinicia el servicio de Docker: sudo systemctl restart docker.
Ejecuta docker ps. Verás que el contenedor sigue vivo gracias al **shim**, evitando tiempo de inactividad en producción.

5. El Pipeline de Seguridad (Aislamiento del Daemon)

El Daemon de Docker corre como root, lo que es un riesgo. El nivel máximo de seguridad aquí es **Rootless Mode**.

- Permite ejecutar todo el stack (dockerd + contenedores) bajo un usuario normal.
- Si un atacante toma el control del motor de Docker, sigue estando atrapado en un usuario sin privilegios del host.

6. Optimización de Recursos y Performance (gRPC)

La comunicación entre dockerd y containerd se realiza mediante **gRPC**. Esto es mucho más rápido que el JSON sobre HTTP tradicional. Para optimizar el rendimiento en clústeres masivos, los arquitectos suelen saltarse dockerd y hablar directamente con containerd para reducir la latencia de creación de contenedores.

7. Persistencia y Redes (Plugins)

A este nivel, el motor no lo hace todo. Utiliza un sistema de **Plugins**:

- **Network Plugins:** Como bridge u overlay.
- **Volume Plugins:** Como local o drivers para la nube (AWS EBS, Azure Disk).
  El motor delega la creación del recurso al plugin y solo gestiona el ciclo de vida.

8. Troubleshooting Forense (Logs del Sistema)

Cuando docker logs no muestra nada, el problema está en el motor:

- **Comando:** journalctl -u docker -f
- **Diagnóstico:** Aquí verás errores de comunicación con containerd o fallos de runc al intentar crear un Namespace (frecuente si el Kernel es muy antiguo o está mal configurado).

9. Anti-Patrones (Errores de Producción)

- **Exponer el Socket TCP sin TLS:** Exponer el puerto 2375 a internet es dar las llaves de root de tu servidor a todo el mundo.
- **Depender de Docker en K8s:** Kubernetes ya no usa Docker como motor por defecto; usa containerd directamente. Un arquitecto debe saber gestionar containerd sin la ayuda de la CLI de Docker.

10. Escalabilidad y Orquestación

En el nivel máximo, entendemos que containerd es el estándar. Al escalar, no pensamos en "instancias de Docker", sino en **Runtimes de Contenedores**. Esto nos permite mezclar nodos que corran Docker con nodos que corran otros motores bajo la misma orquestación.

TOMO II: GESTIÓN DE IMÁGENES Y CONSTRUCCIÓN DE ÉLITE

CAPÍTULO 3: Ciclo de Vida y Anatomía de Capas (UnionFS & CoW)

1. Abstracción de Alto Nivel (Concepto)

Una imagen de Docker no es un archivo único (como un .iso o un .zip), sino un **apilamiento de capas de solo lectura**. Cada instrucción en un Dockerfile crea una nueva capa. Esta estructura permite el "almacenamiento direccionable por contenido": si dos imágenes diferentes usan la misma base de Debian, Docker solo descarga esa base una vez.

- **Analogía:** Imagina una tarta de crêpes. Cada capa es un ingrediente. Si quieres cambiar la mermelada de arriba, no tiras toda la tarta; solo quitas la última capa y pones una nueva. El resto de la tarta permanece intacto.

2. Arquitectura de Bajo Nivel (UnionFS y Copy-on-Write)

El motor de Docker utiliza un sistema de archivos de unión llamado **Overlay2**. Su funcionamiento se divide en tres estados lógicos del Kernel:

- **LowerDir (Capas de la Imagen):** Son capas de "Solo Lectura". Contienen los binarios, librerías y el SO base. Son inmutables.
- **UpperDir (Capa de Escritura):** Cuando lanzas un contenedor, Docker añade una capa delgada de "Lectura/Escritura" en la cima. Todo lo que el contenedor cree o modifique (logs, archivos temporales) vive aquí.
- **Copy-on-Write (CoW):** Si el proceso intenta modificar un archivo que está en una capa inferior (inmutable), el Kernel realiza una copia rápida de ese archivo hacia la **UpperDir** antes de permitir la edición. El archivo original nunca cambia.

3. Diccionario de Comandos "Power User"

Comando	Flag / Uso Pro	Propósito en Nivel Máximo
docker image inspect	--format '{{.RootFS.Layers}}'	Lista los hashes SHA256 de cada capa física en el disco del host.
docker history	--no-trunc	Muestra qué comando exacto del Dockerfile generó cada capa y su tamaño.
docker image prune	-a --filter "until=24h"	Limpieza quirúrgica de imágenes huérfanas que no se han usado en el último día.
docker save/load	-o archivo.tar	Exporta la imagen con todas sus capas para moverla a entornos "Air-gapped" (sin internet).

4. Guía de Implementación Paso a Paso (Auditando Capas)

Vamos a ver cómo Docker optimiza el espacio:

Descarga dos versiones de Nginx:
Bash
docker pull nginx:1.21
docker pull nginx:1.25
Analiza el almacenamiento:
Ejecuta docker system df. Observarás que el espacio total utilizado es mucho menor a la suma de ambas imágenes, porque comparten las capas base de la distribución Linux que utilizan.

Localización física:
En Linux, puedes ver las capas reales en /var/lib/docker/overlay2. Cada carpeta allí corresponde a un SHA que viste en el comando inspect.

5. El Pipeline de Seguridad (Inmutabilidad de Capas)

A nivel máximo, las imágenes deben ser tratadas como **artefactos inmutables**.

- **Content Trust:** Al activar export DOCKER_CONTENT_TRUST=1, Docker verificará la firma digital de cada capa. Si un atacante inyecta código malicioso en una capa de una imagen pública (un ataque de cadena de suministro), Docker se negará a descargarla porque el hash SHA256 no coincidirá con la firma.

6. Optimización de Recursos y Performance (Squashing)

Cada capa añade un pequeño *overhead* al sistema de archivos.

- **Técnica Pro:** Unir comandos con && en el Dockerfile (ej: apt update && apt install && rm -rf /var/lib/apt/lists) asegura que los archivos temporales se borren en la **misma capa** en la que se crearon. Si los borras en una instrucción posterior, el archivo "desaparece" de la vista del contenedor, pero sigue ocupando espacio en la capa inferior de la imagen.

7. Persistencia y Redes (Storage Drivers)

Aunque Overlay2 es el estándar, existen otros drivers como btrfs o zfs para sistemas de archivos avanzados. Un arquitecto elige el driver basado en el rendimiento de I/O. Overlay2 es excelente para lectura rápida, pero para escrituras pesadas, el mecanismo CoW penaliza el rendimiento.

8. Troubleshooting Forense (Capas Fantasma)

- **Escenario:** "He borrado un archivo de 1GB dentro del contenedor pero la imagen sigue pesando lo mismo".
- **Diagnóstico:** El archivo fue creado en una capa inferior. Al "borrarlo", solo has puesto una "marca de borrado" (whiteout) en la capa de escritura superior. El binario sigue ahí, escondido pero ocupando espacio.
- **Solución:** Debes reconstruir la imagen asegurándote de que la creación y el borrado ocurran en el mismo paso del Dockerfile.

9. Anti-Patrones (Errores de Producción)

- **Credenciales en capas intermedias:** Hacer un COPY id_rsa /root/ y luego un RUN rm /root/id_rsa es un error fatal. La clave SSH sigue existiendo en la capa del COPY. Cualquiera con acceso a la imagen puede extraerla explorando las capas.
- **Imágenes "Monolíticas":** Crear imágenes que contienen código, herramientas de build y tests. Esto aumenta la superficie de ataque y el tiempo de despliegue.

10. Escalabilidad y Orquestación

En clústeres masivos (Swarm/K8s), la velocidad de escalado depende del tamaño de las capas. Si tus capas base son pesadas, un nuevo nodo tardará minutos en estar listo. Si usas capas optimizadas y compartidas, el escalado es casi instantáneo porque el nodo ya tiene las capas base en su caché local.

CAPÍTULO 4: Dockerfiles de Alto Rendimiento (Multi-stage & Distroless)

1. Abstracción de Alto Nivel (Concepto)

Construir una imagen no es solo instalar paquetes. El objetivo es crear un artefacto que contenga **únicamente** lo necesario para ejecutar la aplicación en producción. Si tu imagen incluye el compilador (gcc, go, jdk) o el gestor de paquetes (npm, pip) en el entorno final, estás aumentando el peso y la superficie de ataque innecesariamente.

- **Analogía:** Es como enviar un mueble a un cliente. El proceso de fabricación requiere sierras, martillos y pegamento (herramientas de build), pero al cliente solo le envías el mueble terminado (el binario), no la carpintería entera.

2. Arquitectura de Bajo Nivel (El flujo de BuildKit)

Docker utiliza un motor de construcción llamado **BuildKit**. A diferencia del motor antiguo, BuildKit permite:

- **Ejecución en paralelo:** Si tienes etapas independientes en tu Dockerfile, se construyen simultáneamente.
- **Cacheado Inteligente:** Detecta si una capa no ha cambiado y la salta, ahorrando minutos de compilación.
- **Secretos de Build:** Permite usar claves SSH o tokens sin que se graben en el historial de capas de la imagen.

3. Diccionario de Comandos "Power User"

Comando	Parámetro de Élite	Propósito en Nivel Máximo
docker build	--target [stage]	Detiene la construcción en una etapa específica (ideal para ejecutar tests).
docker build	--no-cache	Fuerza una reconstrucción total para asegurar que se descarguen parches de seguridad nuevos.
docker buildx	--platform linux/amd64,linux/arm64	Crea una imagen multi-arquitectura que corre tanto en servidores Intel como en Apple Silicon/Raspberry Pi.
docker image history	--human	Analiza el tamaño de cada instrucción para detectar "capas gordas" que deban optimizarse.

4. Guía de Implementación Paso a Paso (El Dockerfile Perfecto)

Este es el estándar de oro para una aplicación en Go o Node.js:

Dockerfile

```
# ETAPA 1: Build (Carpintería)
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
# Compilación estática (sin dependencias externas)
RUN CGO_ENABLED=0 GOOS=linux go build -o myapp main.go

# ETAPA 2: Producción (El Mueble)
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app/myapp /myapp
USER nonroot:nonroot
ENTRYPOINT ["/myapp"]
```

5. El Pipeline de Seguridad (Imágenes Distroless)

El nivel máximo de blindaje se logra con **Distroless**. Son imágenes que no contienen una shell (bash/sh), ni gestores de paquetes, ni herramientas de diagnóstico.

- **Beneficio:** Si un hacker logra explotar tu aplicación, no encontrará una terminal para ejecutar comandos ni podrá descargar malware mediante curl o wget, porque simplemente **no existen** en la imagen.

6. Optimización de Recursos y Performance (Contexto de Build)

El .dockerignore es tu mejor amigo. Si no lo usas, Docker enviará todo tu directorio (incluyendo carpetas pesadas como node_modules o archivos .git) al Daemon de Docker antes de empezar el build. Un arquitecto siempre filtra el contexto para que el build sea instantáneo.

7. Persistencia y Redes (Montajes de Caché)

BuildKit permite usar montajes de tipo caché:

```
RUN --mount=type=cache,target=/root/.cache/go-build go build ...
```

Esto mantiene la caché de compilación entre diferentes builds, reduciendo el tiempo de espera de 5 minutos a 10 segundos en desarrollos iterativos.

8. Troubleshooting Forense (Debug de Stages)

- **Escenario:** "La aplicación falla en la etapa final pero en el build parece que todo está bien".
- **Estrategia Pro:** Usa --target builder para entrar en la etapa de compilación:
  docker build --target builder -t debug-image .
  Luego lanza un contenedor interactivo de esa imagen para inspeccionar los binarios generados antes de que se copien a la imagen limpia.

9. Anti-Patrones (Errores de Producción)

- **Múltiples instrucciones RUN:** Cada RUN es una capa. Unir comandos con && es obligatorio, pero con Multi-stage esto es menos crítico ya que la imagen final solo tendrá las capas de la última etapa.
- **No usar el usuario USER:** Ejecutar el proceso final como root invalida el 80% de las capas de seguridad de Docker.

10. Escalabilidad y Orquestación

Imágenes pequeñas = Escalado rápido. En Kubernetes, si un nodo muere y tiene que descargar una imagen de 1GB, el servicio estará caído durante minutos. Si la imagen pesa 15MB (Distroless), el servicio vuelve a estar en línea en segundos.

TOMO III: OPERACIONES, REDES Y PERSISTENCIA

CAPÍTULO 5: Redes en Docker: Del Bridge a las redes Overlay

1. Abstracción de Alto Nivel (Concepto)

En Docker, la red es virtualizada. El motor crea un stack de red aislado para cada contenedor, permitiendo que varios procesos escuchen en el mismo puerto (ej. el 80) dentro de sus propios entornos sin entrar en conflicto.

- **Analogía:** Es como un complejo de oficinas. Cada oficina tiene su propia extensión interna (IP del contenedor). El recepcionista (Docker Bridge) sabe a qué extensión enviar cada llamada que llega desde la calle (Puerto del Host).

2. Arquitectura de Bajo Nivel (Iptables y veth pairs)

El aislamiento de red se basa en el **Network Namespace**. Cuando creas un contenedor, ocurren tres eventos a nivel de Kernel:

1. **veth pair:** Se crea un "cable virtual". Un extremo se conecta al contenedor (eth0) y el otro al bridge del host (vethXXX).
2. **Bridge (docker0):** Un switch virtual basado en software que actúa como puerta de enlace (Gateway).
3. **Iptables / NAT:** Docker inserta reglas en el firewall del host para que el tráfico que llega al puerto 8080 del servidor físico sea redirigido (DNAT) a la IP privada del contenedor.

3. Diccionario de Comandos "Power User"

Comando	Driver / Flag	Uso en Nivel Máximo
docker network create	--driver overlay	Crea una red que abarca múltiples hosts (necesario para Swarm/K8s).
docker run	--network host	Elimina el aislamiento; el contenedor usa la IP y puertos del host directamente (máximo rendimiento).
docker network inspect	[nombre]	Muestra el JSON con las IPs asignadas y los IDs de los endpoints conectados.
docker run	--link	**(Anti-patrón)** Obsoleto. En nivel máximo usamos DNS interno por nombre de contenedor.

4. Guía de Implementación Paso a Paso (Redes Personalizadas)

Nunca uses la red por defecto bridge en producción, ya que no permite resolución de nombres por DNS.

Crear red aislada:
Bash
docker network create --driver bridge mi_red_segura
Lanzar servicios:
Bash
docker run -d --name db --network mi_red_segura redis
docker run -d --name web --network mi_red_segura nginx
Prueba de DNS:
docker exec -it web ping db. Docker resolverá la IP de db automáticamente sin que necesites saberla.

5. El Pipeline de Seguridad (Aislamiento Total)

El nivel de seguridad máximo es la **Red Interna**. Puedes crear redes que no tengan salida a internet (--internal).

- **Estrategia:** Pon tu base de datos en una red interna y tu servidor web en dos redes (la interna para hablar con la DB y una pública para internet). Así, si hackean el servidor web, no pueden usar la DB para descargar herramientas externas.

6. Optimización de Recursos y Performance

Cada salto en el bridge virtual añade una latencia mínima (microsegundos).

- **High Performance:** Para aplicaciones de trading o streaming de video con latencia ultra-baja, usamos --network host. El proceso se une al stack de red del host, saltándose el bridge y el NAT de iptables.

7. Persistencia y Redes (Service Discovery)

Docker incluye un servidor DNS embebido en el motor. Cuando un contenedor busca a otro por su nombre, el motor intercepta la petición DNS antes de que salga al host. Esto es la base del **Service Discovery** en arquitecturas de microservicios.

8. Troubleshooting Forense

- **Escenario:** "Puedo hacer ping por IP, pero no por nombre".
- **Diagnóstico:** Estás usando la red bridge por defecto de Docker. Esa red no tiene activado el servidor DNS interno por diseño de seguridad heredado.
- **Solución:** Mueve los contenedores a una *User-Defined Network*.

9. Anti-Patrones (Errores de Producción)

- **Publicar todos los puertos:** Usar -p 80:80 es correcto, pero publicar puertos de bases de datos al mundo (-p 5432:5432) es un riesgo innecesario. Solo expón lo que el usuario final debe tocar.
- **IPs estáticas manuales:** Intentar asignar IPs fijas a contenedores (--ip 172.18.0.10). Es un error de arquitectura; usa nombres de servicio y deja que Docker gestione el direccionamiento.

10. Escalabilidad y Orquestación

En el nivel máximo, saltamos a las redes **Overlay**. Usan protocolos como **VXLAN** para encapsular paquetes y enviarlos a través de diferentes servidores físicos. Esto permite que un contenedor en el Servidor A hable con uno en el Servidor B como si estuvieran en el mismo switch.

CAPÍTULO 6: El Triángulo de la Persistencia (Volumes, Bind Mounts y Tmpfs)

1. Abstracción de Alto Nivel (Concepto)

En el nivel máximo de ingeniería, separamos el **Estado** (los datos) de la **Lógica** (el contenedor). El almacenamiento en Docker no es una característica única, sino una tríada de opciones diseñadas para diferentes casos de uso: producción, desarrollo y seguridad.

- **Analogía:** El contenedor es como un reproductor de DVD (lógica). Los datos son el disco DVD (estado). Puedes cambiar el reproductor por uno más moderno, pero mientras mantengas el disco, la película (tus datos) seguirá ahí.

2. Arquitectura de Bajo Nivel (El Bypass del UnionFS)

Como vimos en el Capítulo 3, escribir en la capa del contenedor (*Copy-on-Write*) es lento. Los mecanismos de persistencia funcionan creando un **montaje (mount)** que se salta el sistema de archivos de unión de Docker:

- **Volumes:** Docker crea y gestiona un directorio en /var/lib/docker/volumes/. Es el método recomendado porque Docker tiene control total sobre los permisos y la gestión del ciclo de vida.
- **Bind Mounts:** Mapeas una ruta específica de tu host (ej. /home/user/app) directamente al contenedor. Es el host quien manda.
- **Tmpfs:** El montaje ocurre directamente en la **memoria RAM** del host. Nunca toca el disco.

3. Diccionario de Comandos "Power User"

Comando	Flag Avanzado	Propósito en Nivel Máximo
docker volume create	--driver [nombre]	Permite usar drivers de terceros para persistir en la nube (NFS, Azure, AWS).
docker run	--mount type=volume	La sintaxis moderna y explícita que sustituye a -v (evita errores de rutas).
docker run	--tmpfs /app/cache	Almacena datos volátiles en RAM para máxima velocidad y seguridad.
docker volume prune	*(Sin flags)*	Elimina todos los volúmenes que no están siendo usados por ningún contenedor.

4. Guía de Implementación Paso a Paso (Persistencia de Bases de Datos)

Nunca dejes que una base de datos escriba en la capa del contenedor.

Crear un volumen gestionado:
Bash
```
docker volume create pg_data_prod
```
Lanzar el contenedor usando la sintaxis --mount (Nivel Pro):
Bash
```
docker run -d --name db_master \
  --mount source=pg_data_prod,target=/var/lib/postgresql/data \
  postgres:15
```
Verificación de independencia:
Borra el contenedor (docker rm -f db_master). Al crear uno nuevo apuntando al mismo volumen, verás que todas tus tablas siguen ahí.

5. El Pipeline de Seguridad (Protección de Datos)

El uso de **Montajes de Solo Lectura** es una técnica de hardening vital.

- **Escenario:** Tienes un archivo de configuración (config.json) en el host que el contenedor debe leer.
- **Implementación:** docker run --mount type=bind,source=$(pwd)/config.json,target=/app/config.json,readonly nginx.
  Esto impide que, si el contenedor es comprometido, el atacante pueda modificar la configuración del servicio desde dentro.

6. Optimización de Recursos y Performance (I/O Nativo)

Al usar volúmenes o bind mounts, el proceso del contenedor escribe directamente en el sistema de archivos del host a **velocidad nativa**. Se elimina el overhead del driver overlay2. Para logs masivos o bases de datos de alta transaccionalidad, este es el único camino aceptable.

7. Persistencia y Redes (Almacenamiento Distribuido)

En clústeres (Swarm/K8s), un volumen local no sirve si el contenedor salta de un Servidor A a un Servidor B. Aquí es donde entran los **Volume Drivers de Red**. El volumen se monta mediante protocolos como NFS o iSCSI, permitiendo que los datos "sigan" al contenedor por toda la red.

8. Troubleshooting Forense

- **Escenario:** "He borrado el contenedor y el volumen, pero el disco sigue lleno".
- **Diagnóstico:** Los volúmenes en Docker son persistentes por diseño. Si usas -v al correr un contenedor y no lo borras explícitamente, se quedan como **"Orphaned Volumes"**.
- **Solución:** Ejecuta docker volume inspect para verificar qué volúmenes no tienen Labels de uso y límpialos.

9. Anti-Patrones (Errores de Producción)

- **Escribir logs en el contenedor:** Llenarás la capa de escritura y bloquearás el host. Usa siempre un volumen o redirige a stdout/stderr.
- **Bind Mounts en producción:** Depender de rutas fijas del host (/home/juan/data) rompe la portabilidad. Si mueves el contenedor a otro servidor donde no existe "juan", el sistema fallará. Usa **Volumes** siempre que sea posible.

10. Escalabilidad y Orquestación

En el nivel de Arquitecto de Kubernetes (que veremos más adelante), los volúmenes se convierten en **Persistent Volume Claims (PVC)**. Entender la diferencia entre un montaje local y uno gestionado por el motor es la base para configurar el almacenamiento en la nube sin perder datos.

TOMO IV: ORQUESTACIÓN CON DOCKER SWARM

CAPÍTULO 7: Swarm Mode: Servicios, Escalabilidad y Healthchecks

1. Abstracción de Alto Nivel (Concepto)

Docker Swarm transforma un grupo de servidores independientes en un único **Recurso Virtual**. Ya no despliegas un contenedor en el "Servidor A", sino que le pides al clúster que ejecute un "Servicio" con N réplicas, y el orquestador decide dónde colocarlas basándose en la carga.

- **Analogía:** Si un contenedor es un músico solista, Swarm es el **Director de Orquesta**. Si un músico se cansa o se va (falla el nodo), el director hace que otro músico empiece a tocar la misma partitura para que la música no pare.

2. Arquitectura de Bajo Nivel (Manager vs Worker)

Swarm utiliza un modelo de estado deseado y una arquitectura de consenso basada en el algoritmo **Raft**:

- **Manager Nodes:** Son el cerebro. Mantienen el estado del clúster, gestionan la programación de tareas y mantienen el quórum. En nivel máximo, siempre usamos un número impar (3 o 5) para evitar el "Split-Brain".
- **Worker Nodes:** Son el músculo. Solo ejecutan las tareas (contenedores) que el Manager les asigna.
- **Ingress Mesh:** Una capa de red interna que redirige el tráfico. Si pides el puerto 80 a cualquier nodo del clúster, Swarm te llevará al contenedor correcto, aunque no esté en ese nodo físico.

3. Diccionario de Comandos "Power User"

Comando	Flag Avanzado	Propósito en Nivel Máximo
docker swarm init	--advertise-addr	Define qué IP usar para la comunicación interna del clúster (crítico en multi-interface).
docker service create	--replicas 5	Define el estado deseado: Swarm mantendrá siempre 5 procesos vivos.
docker service update	--image [img]:v2	Realiza un **Rolling Update** (actualiza de uno en uno sin caída de servicio).
docker service scale	web=10	Escala la infraestructura en segundos para absorber picos de tráfico.

4. Guía de Implementación Paso a Paso (El Servicio Resiliente)

1. **Iniciación:** docker swarm init.

Despliegue con Healthcheck (Nivel Pro):
Bash
```
docker service create --name app_web \
  --replicas 3 \
  --health-cmd "curl -f http://localhost/ || exit 1" \
  --health-interval 5s \
  --publish published=80,target=80 \
  nginx
```
Simulación de Fallo: Borra manualmente un contenedor con docker rm -f. Verás cómo Swarm lo recrea instantáneamente para cumplir el "estado deseado".

5. El Pipeline de Seguridad (MTLS Automático)

A diferencia de otros orquestadores, Swarm activa por defecto el cifrado **mTLS (Mutual TLS)**. Cada nodo que se une al clúster recibe un certificado digital. Toda la comunicación entre managers y workers está cifrada de extremo a extremo automáticamente, rotando los certificados cada 90 días sin intervención humana.

6. Optimización de Recursos y Performance (Rolling Updates)

Para evitar que una actualización rompa el servicio, configuramos el paralelismo:

```
docker service update --update-parallelism 2 --update-delay 10s app_web
```

Esto actualiza los contenedores de dos en dos, esperando 10 segundos entre cada par para confirmar que están sanos. Si la nueva versión falla, Swarm puede hacer un **Rollback** automático a la versión anterior.

7. Persistencia y Redes (Overlay Routing)

Swarm utiliza la red **Overlay**. Cuando creas un servicio, Docker asigna una **VIP (Virtual IP)** al servicio. El balanceador de carga interno del kernel (IPVS) distribuye las conexiones entrantes entre todas las réplicas de forma transparente.

8. Troubleshooting Forense

- **Escenario:** "He actualizado el servicio pero los cambios no se ven".
- **Diagnóstico:** El servicio puede estar en estado REJECTED.
- **Comando Forense:** docker service ps app_web --no-trunc. Aquí verás el error real (ej: falta de memoria en los nodos o imagen no encontrada en el registro).

9. Anti-Patrones (Errores de Producción)

- **Número par de Managers:** Tener 2 managers es peor que tener 1. Si pierden conexión entre ellos, ninguno puede tomar decisiones porque no hay mayoría (quórum).
- **Usar docker run en un clúster:** En Swarm, los contenedores individuales no son gestionados. Usa siempre docker service o docker stack.

10. Escalabilidad y Orquestación

Swarm es excelente por su sencillez y su integración nativa con Docker Compose. Es la opción ideal para arquitecturas que necesitan alta disponibilidad sin la complejidad extrema de Kubernetes, permitiendo gestionar miles de contenedores con una fracción del esfuerzo operativo.

CAPÍTULO 8: Gestión de Secretos y Configuración en Producción

1. Abstracción de Alto Nivel (Concepto)

Un **Secret** es un objeto de datos sensible (como una clave SSH o un password de DB) que Swarm almacena de forma cifrada. A diferencia de las variables de entorno, los secretos no se guardan en el disco de los nodos trabajadores; se transmiten por la red cifrada y se montan en un sistema de archivos temporal en la memoria RAM del contenedor.

- **Analogía:** Una variable de entorno es como escribir la clave del Wi-Fi en un post-it pegado a la pantalla. Un **Secret** es una caja fuerte digital que solo se abre dentro de la habitación del contenedor cuando este empieza a trabajar, y desaparece en cuanto el contenedor se va.

2. Arquitectura de Bajo Nivel (El Ciclo de Vida del Secreto)

La seguridad de los secretos en Swarm es superior debido a su integración con el Kernel:

1. **Cifrado en Reposo:** Los secretos se guardan en el log de Raft de los Managers, cifrados en el disco.
2. **Transmisión mTLS:** Se envían a los nodos Workers a través de canales TLS mutuos.
3. **Montaje en RAM (tmpfs):** El secreto aparece dentro del contenedor en /run/secrets/<nombre_del_secreto>. Es un sistema de archivos volátil; si el nodo pierde corriente, el secreto desaparece de la memoria.

3. Diccionario de Comandos "Power User"

Comando	Acción Pro	Propósito en Nivel Máximo
openssl rand -base64 20 | docker secret create db_pass -	Creación segura	Genera un secreto aleatorio y lo inyecta directamente sin dejar rastro en el historial de bash.
docker secret ls	Listado	Muestra los metadatos de los secretos, pero **nunca** el contenido.
docker service update --secret-rm --secret-add	Rotación	Permite cambiar una contraseña en caliente sin borrar el servicio.
docker config create	Configuración	Similar al secreto pero para datos no sensibles (archivos .conf, .yml).

4. Guía de Implementación Paso a Paso (Inyección de Secretos)

Crear el secreto:

Bash

```
echo "password_ultra_seguro_2024" | docker secret create mi_db_password -
```

Vincularlo a un servicio:

Bash

```
docker service create --name mi_app \
  --secret mi_db_password \
  -e POSTGRES_PASSWORD_FILE=/run/secrets/mi_db_password \
  postgres
```

Auditoría interna:

Ejecuta docker exec $(docker ps -q) ls /run/secrets/. Verás el archivo, pero si intentas hacer un env en la consola, la contraseña no aparecerá allí.

5. El Pipeline de Seguridad (Rotación de Secretos)

El nivel de élite implica que las contraseñas no son eternas. Swarm permite la **rotación sin downtime**:

- Actualizas el servicio quitando el secreto viejo y añadiendo el nuevo.
- Swarm realiza un rolling update, reemplazando los contenedores uno a uno con la nueva credencial.

6. Optimización de Recursos y Performance (Configs vs Secrets)

Usa **Docker Configs** para archivos de configuración pesados (como un nginx.conf de 500 líneas). A diferencia de los secretos, las *configs* no consumen memoria cifrada de la misma forma, lo que libera recursos en el plano de control (Manager) mientras mantienes la inmutabilidad de la configuración.

7. Persistencia y Redes (Propagación)

Los secretos solo se propagan a los nodos que tienen tareas asignadas de ese servicio específico. Si un nodo Worker es comprometido y no está ejecutando el servicio de la Base de Datos, el atacante no encontrará rastro del secreto de la DB en ese nodo.

8. Troubleshooting Forense

- **Escenario:** "Mi aplicación dice que el archivo de secreto no existe".
- **Diagnóstico:** Los secretos se montan por defecto con permisos 0444 (solo lectura) y pertenecen al usuario root.
- **Solución de Arquitecto:** Si tu app corre con un usuario no-root, debes definir el UID/GID al crear el servicio:
  --secret source=mi_pass,target=pass,uid=1000,gid=1000.

9. Anti-Patrones (Errores de Producción)

- **Pasar contraseñas por -e (Variables de entorno):** Cualquier persona con acceso al comando docker inspect o cualquier sistema de monitoreo de logs podrá ver la contraseña en texto plano.
- **Guardar secretos en el Dockerfile:** Como aprendimos en el Tomo II, las capas son eternas. Un secreto en un Dockerfile es una vulnerabilidad permanente.

10. Escalabilidad y Orquestación

En el nivel máximo, combinamos **Docker Stack** con archivos YAML para gestionar decenas de secretos y configuraciones de forma declarativa. Este es el puente directo hacia los *ConfigMaps* y *Secrets* de Kubernetes, que funcionan bajo una lógica casi idéntica.

TOMO V: KUBERNETES – EL ESTÁNDAR DE ORQUESTACIÓN

CAPÍTULO 9: Arquitectura de K8s: El Control Plane y los Nodos

1. Abstracción de Alto Nivel (Concepto)

Kubernetes es un **Sistema Operativo para el Centro de Datos**. Si Docker aísla procesos y Swarm gestiona servicios, Kubernetes gestiona el ciclo de vida completo de aplicaciones complejas, auto-reparándose, escalando según la carga y gestionando el tráfico de red de forma declarativa.

- **Analogía:** Si Docker es un coche y Swarm es una flota de taxis, Kubernetes es el **Control de Tráfico Aéreo** de una ciudad entera: sabe dónde está cada avión, garantiza que no choquen y redirige los vuelos si una pista (nodo) se bloquea.

2. Arquitectura de Bajo Nivel (El Cerebro y el Músculo)

Kubernetes se divide en dos secciones críticas que un Arquitecto debe conocer para depurar fallos:

- **Control Plane (El Cerebro):**
  - **etcd:** La base de datos clave-valor donde se guarda TODA la verdad del clúster. Si etcd muere, el clúster pierde la memoria.
  - **API Server:** El único punto de entrada. Todo (incluso los componentes internos) habla con él mediante REST.
  - **Scheduler:** El "logístico". Decide en qué nodo encaja cada carga de trabajo según los recursos disponibles.
  - **Controller Manager:** El "vigilante". Si pides 3 réplicas y hay 2, él da la orden de crear la que falta.
- **Worker Nodes (El Músculo):**
  - **Kubelet:** El agente que corre en cada servidor. Se asegura de que los contenedores estén corriendo según lo ordenado.
  - **Kube-proxy:** El ingeniero de redes. Gestiona las reglas de IP para que el tráfico llegue al contenedor correcto.

3. Diccionario de Comandos "Power User"

Comando	Parámetro de Élite	Propósito en Nivel Máximo
kubectl get pods	-o wide	Muestra en qué nodo físico exacto y con qué IP interna está corriendo el Pod.
kubectl explain	pod.spec.containers	Documentación técnica interactiva desde la terminal para conocer cada campo del YAML.
kubectl top nodes	*(Requiere metrics-server)*	Muestra el consumo real de CPU/RAM de los servidores físicos en tiempo real.
kubectl config	get-contexts	Permite saltar entre diferentes clústeres (Desarrollo, Prod, Cloud).

4. Guía de Implementación Paso a Paso (Tu primer Pod Pro)

En K8s no usamos docker run. Usamos archivos YAML o comandos imperativos para generar el estado:

Crear un Pod rápidamente:
Bash
kubectl run mi-web --image=nginx:alpine --labels="env=prod,app=frontend"
Inspección Forense:
kubectl describe pod mi-web. Aquí verás los "Events", que te dicen exactamente qué pasó desde que pediste el Pod hasta que arrancó.

Acceso Interno:
kubectl port-forward mi-web 8080:80. Crea un túnel temporal desde tu portátil al clúster para testeo.

5. El Pipeline de Seguridad (RBAC y Namespaces)

El nivel máximo de seguridad en K8s es el **Control de Acceso Basado en Roles (RBAC)**.

- **Namespaces:** Dividen el clúster en "parcelas". Puedes tener un Namespace de ventas y otro de pagos.
- **Aislamiento:** Un desarrollador con acceso al Namespace de ventas no puede ni siquiera ver los Pods que corren en pagos.

6. Optimización de Recursos y Performance (Requests & Limits)

Un Arquitecto nunca despliega un Pod sin límites.

- **Requests:** Lo que el Pod tiene garantizado (reserva de recursos).
- **Limits:** El techo máximo. Si el Pod intenta usar más CPU de la limitada, K8s lo "estrangula" (throttling). Si usa más RAM de la limitada, lo mata (OOMKilled).

7. Persistencia y Redes (El Pod Network)

A diferencia de Docker, en K8s **todos los Pods pueden hablar con todos los demás Pods** sin necesidad de NAT, sin importar en qué servidor estén. Esto se logra mediante plugins de red (CNI) como Calico o Flannel.

8. Troubleshooting Forense

- **Escenario:** El Pod está en ImagePullBackOff.
- **Diagnóstico:** El Kubelet no puede descargar la imagen (error de red, credenciales del registro mal puestas o nombre de imagen equivocado).
- **Comando:** kubectl logs [nombre-pod] para ver errores de la aplicación, o kubectl describe para errores de infraestructura.

9. Anti-Patrones (Errores de Producción)

- **Usar Pods sueltos:** Los Pods son efímeros. Si un nodo muere, el Pod muere. Un arquitecto usa siempre **Deployments** para que K8s garantice la recreación automática.
- **Almacenar datos en el Pod:** Si el Pod se reinicia, los datos se borran. Usa **Persistent Volumes**.

10. Escalabilidad y Orquestación

K8s escala a miles de nodos. Mediante el **Horizontal Pod Autoscaler (HPA)**, el clúster puede observar el uso de CPU y, si supera el 70%, crear automáticamente más réplicas de tu aplicación para absorber el tráfico.

CAPÍTULO 10: Deployments, Pods y Replicasets: Control de Estado

1. Abstracción de Alto Nivel (Concepto)

En Kubernetes, casi nunca gestionamos Pods individuales. Utilizamos un **Deployment**, que actúa como un gerente de planta. Tú no le dices "crea un contenedor", le dices "quiero que siempre haya 3 réplicas de este servicio funcionando". Si una instancia falla, el Deployment se encarga de que otra nazca automáticamente.

- **Analogía:** Si un Pod es un empleado, el **Deployment** es el contrato de servicios. Si el empleado se pone enfermo, el contrato obliga a la empresa a poner un sustituto inmediatamente para que el trabajo no se detenga.

2. Arquitectura de Bajo Nivel (El bucle de control)

El Deployment no crea Pods directamente. Utiliza un objeto intermedio llamado **ReplicaSet**:

1. **Deployment:** Gestiona las versiones (v1, v2) y las estrategias de actualización.
2. **ReplicaSet:** Se asegura de que el número de réplicas sea el correcto mediante **Label Selectors**.
3. **Pod:** La unidad mínima que contiene tus contenedores.

3. Diccionario de Comandos "Power User"

Comando	Parámetro de Élite	Propósito en Nivel Máximo
kubectl create deploy	--image --dry-run=client -o yaml	Genera la estructura del YAML sin crear nada, para que puedas editarlo antes de aplicarlo.
kubectl scale	deploy/nombre --replicas=5	Cambia el número de instancias en vuelo instantáneamente.
kubectl rollout undo	deployment/nombre	El "botón del pánico": vuelve a la versión anterior si la nueva falla.
kubectl get rs	*(Cualquiera)*	Permite ver los ReplicaSets antiguos que K8s guarda para poder hacer rollback.

4. Guía de Implementación Paso a Paso (Escalado de Élite)

Basado en tus apuntes de **"Scaling Replicasets"**:

Crear el despliegue:
Bash
kubectl create deployment mi-apache --image=httpd
Escalar a nivel de producción:
Bash
kubectl scale deployment mi-apache --replicas=3
Verificación de alta disponibilidad:
Usa kubectl get pods -o wide para confirmar que K8s ha distribuido los Pods en diferentes nodos físicos para evitar que un fallo de hardware tumbe todo el servicio.

5. El Pipeline de Seguridad (Immutable Desired State)

A diferencia de Docker Swarm, en K8s el estándar de seguridad es el **Despliegue Declarativo**.

- **Técnica:** Nunca uses kubectl edit en vivo. Usa archivos YAML y el comando kubectl apply -f archivo.yaml.
- **Beneficio:** Esto permite que tu infraestructura esté versionada en Git (GitOps). Si alguien borra algo por error, simplemente vuelves a aplicar el archivo y K8s restaura el estado seguro.

6. Optimización de Recursos y Performance (Estrategias de Reemplazo)

Un Arquitecto decide cómo se actualiza la app:

- **RollingUpdate:** (Por defecto) Va matando uno viejo y creando uno nuevo. No hay caída de servicio.
- **Recreate:** Mata todos los viejos primero y luego crea los nuevos. Útil cuando la base de datos no soporta dos versiones de la app hablando a la vez.

7. Persistencia y Redes (Labels & Selectors)

El "pegamento" de K8s son las **Labels**. Un ReplicaSet no "posee" Pods, sino que busca Pods que tengan una etiqueta específica (ej. app: nginx). Si cambias la etiqueta de un Pod en caliente, el ReplicaSet creerá que ha perdido una réplica y creará una nueva, "liberando" al Pod original para que lo investigues (técnica de cuarentena).

8. Troubleshooting Forense

- **Escenario:** "He escalado a 10 réplicas pero solo hay 2 funcionando".
- **Diagnóstico:** El clúster se ha quedado sin recursos físicos.
- **Inspección de Nivel Dios:**
  kubectl describe deployment [nombre].
  Busca en la sección de "Events". Verás mensajes como FailedScheduling: el Scheduler no encuentra un nodo con suficiente RAM/CPU para tus nuevas réplicas.

9. Anti-Patrones (Errores de Producción)

- **No configurar Readiness Probes:** Si no le dices a K8s cómo saber si la app está lista, enviará tráfico al Pod en cuanto el contenedor arranque, incluso si la app aún se está cargando, dando errores a los usuarios.
- **Nombres duplicados:** Como apuntaste, no puedes tener dos recursos del mismo tipo con el mismo nombre en el mismo Namespace.

10. Escalabilidad y Orquestación

En el nivel máximo, el escalado no es manual. Usamos el **HPA (Horizontal Pod Autoscaler)**. Este componente monitoriza el consumo de CPU de tus Pods y, según tus notas de "Service Update", ajusta las réplicas automáticamente entre un mínimo y un máximo definido por ti.

TOMO VI: SEGURIDAD, HARDENING Y CI/CD (NIVEL DIOS)

CAPÍTULO 11: Networking en K8s: ClusterIP, NodePort y LoadBalancer

1. Abstracción de Alto Nivel (Concepto)

Un **Service** en Kubernetes es una abstracción que define un grupo lógico de Pods y una política para acceder a ellos. Es la "IP fija" o el nombre DNS que nunca cambia, actuando como un balanceador de carga interno.

- **Analogía:** Imagina un restaurante con 5 camareros (Pods). Los clientes no llaman a un camarero específico por su nombre, van al **Mostrador de Entrada (Service)**. El mostrador recibe la petición y la asigna al camarero que esté libre en ese momento.

2. Arquitectura de Bajo Nivel (Kube-proxy e IPVS)

Cuando creas un Service, no se crea un proceso real, sino una regla en la red del clúster gestionada por el **Kube-proxy**:

- **Endpoint Slice:** K8s mantiene una lista actualizada de las IPs de los Pods que tienen las etiquetas (labels) correctas.
- **Iptables/IPVS:** El Kube-proxy configura el Kernel de cada nodo para que, cuando llegue tráfico a la IP del Service, se redirija automáticamente a uno de los Pods sanos.

3. Diccionario de Comandos "Power User"

Comando	Parámetro de Élite	Propósito en Nivel Máximo
kubectl expose	--type=NodePort --port=80	Crea un Service de forma rápida a partir de un Deployment existente.
kubectl get svc	-o wide	Muestra la IP virtual (ClusterIP) y los puertos mapeados.
kubectl explain svc.spec	--recursive	Explica todos los campos posibles para configurar el servicio (como vimos en tus notas).
kubectl get endpoints	*(Nombre del SVC)*	Muestra las IPs reales de los Pods a los que el servicio está enviando tráfico.

4. Guía de Implementación Paso a Paso (Tipos de Servicio)

Dependiendo de quién necesite acceder a la app, elegimos un tipo:

1. **ClusterIP (Por defecto):** Solo accesible desde dentro del clúster. Ideal para Bases de Datos.
2. **NodePort:** Abre un puerto estático (rango 30000-32767) en cada IP de cada Servidor (Nodo). Permite acceso externo básico.
3. **LoadBalancer:** El estándar en la nube (AWS/Azure/GCP). Crea un balanceador real fuera del clúster con una IP pública única.

5. El Pipeline de Seguridad (Aislamiento de Red)

En nivel máximo, no basta con el Service. Usamos **Network Policies**.

- Por defecto, K8s permite que todos hablen con todos.
- Con una Network Policy, puedes decir: "El Pod de Base de Datos **solo** acepta tráfico en el puerto 5432 proveniente del Pod con la etiqueta role: backend". Cualquier otro intento de conexión será bloqueado por el Kernel.

6. Optimización de Recursos y Performance (Session Affinity)

Si tu aplicación necesita que un usuario siempre hable con el mismo Pod (para mantener la sesión en memoria), usamos:

service.spec.sessionAffinity: ClientIP.

Esto hace que K8s use la IP del cliente para dirigirlo siempre a la misma réplica durante un tiempo determinado.

7. Persistencia y Redes (DNS CoreDNS)

Kubernetes tiene un servicio interno llamado **CoreDNS**. Cada vez que creas un servicio llamado mi-api, K8s registra el nombre DNS mi-api.default.svc.cluster.local.

- **Ventaja:** Tus microservicios no necesitan saber IPs; solo configuras el backend para que busque http://mi-api.

8. Troubleshooting Forense

- **Escenario:** "El servicio existe pero el comando curl da timeout".
- **Diagnóstico:** Los **Selectors** están mal. El servicio busca Pods con una etiqueta que no existe.
- **Comando Forense:** kubectl describe svc [nombre]. Si en la sección de Endpoints pone <none>, el Service está vacío; no ha encontrado ningún Pod que coincida con sus etiquetas.

9. Anti-Patrones (Errores de Producción)

- **NodePort masivos:** Abrir puertos NodePort para todo. Expone la IP de tus servidores físicos, lo cual es un riesgo de seguridad. Usa **Ingress** (lo veremos en el siguiente capítulo) para centralizar el tráfico.
- **Puertos mal mapeados:** Confundir port (puerto del servicio) con targetPort (puerto donde escucha el contenedor).

10. Escalabilidad y Orquestación

Los servicios son elásticos. Si escalas tu Deployment de 3 a 100 réplicas, el Service detectará automáticamente las 97 nuevas IPs y empezará a enviarles tráfico sin que tengas que actualizar ni una sola línea de configuración de red.

CAPÍTULO 12: Escaneo de Vulnerabilidades y Firmado de Imágenes

1. Abstracción de Alto Nivel (Concepto)

La seguridad en contenedores no es un evento único, es un proceso continuo. La mayoría de las imágenes públicas contienen vulnerabilidades conocidas (CVE). El **Hardening** consiste en reducir la "superficie de ataque": cuanto menos software haya dentro de la imagen, menos agujeros podrá encontrar un atacante.

- **Analogía:** Si tu imagen es una fortaleza, el escaneo de vulnerabilidades es como contratar a un detective que busca grietas en las paredes cada día. El firmado de imágenes es el sello de cera en una carta: garantiza que nadie la ha abierto ni manipulado desde que salió del remitente.

2. Arquitectura de Bajo Nivel (Base de Datos de CVEs)

Herramientas como **Docker Scout** o **Trivy** no analizan el código línea por línea, sino que:

1. **Indexan las capas:** Generan un inventario (SBOM - Software Bill of Materials) de todas las librerías y binarios (ej: openssl 1.1.1).
2. **Cotejan con la NVD:** Comparan esa lista con la *National Vulnerability Database*.
3. **Calculan el riesgo:** Clasifican los fallos en *Critical, High, Medium o Low* basándose en la facilidad de explotación.

3. Diccionario de Comandos "Power User"

Comando	Acción Pro	Propósito en Nivel Máximo
docker scout quickview	Resumen rápido	Da una visión general de las vulnerabilidades sin entrar en detalles.
docker scout cves [img]	Listado detallado	Muestra exactamente qué capa y qué librería tiene el fallo de seguridad.
docker scout recommendations	Guía de solución	Te dice exactamente a qué imagen base debes actualizar para eliminar los fallos.
docker trust inspect	Verificación	Comprueba si la imagen que vas a bajar tiene una firma digital válida.

4. Guía de Implementación Paso a Paso (Auditando una imagen)

Vamos a auditar una imagen de nginx antigua para ver el peligro:

Análisis inicial:
Bash
docker scout quickview nginx:1.17
(Verás decenas de vulnerabilidades críticas porque es una versión obsoleta).

Búsqueda de la "capa culpable":
Bash
docker scout cves nginx:1.17
Aplicación de parches: Sigue la recomendación de Scout para actualizar a la versión stable-alpine, que reduce las vulnerabilidades a casi cero.

5. El Pipeline de Seguridad (Docker Content Trust)

El nivel máximo de confianza se activa con **DCT (Docker Content Trust)**.

- **Activación:** export DOCKER_CONTENT_TRUST=1.
- **Efecto:** A partir de este momento, el motor de Docker se negará a descargar o ejecutar cualquier imagen que no haya sido firmada digitalmente por su autor. Si un hacker sube una imagen maliciosa con el mismo nombre a Docker Hub, tu sistema la bloqueará.

6. Optimización de Recursos y Performance (VEX)

El escaneo puede ser lento si se hace en cada build. Los arquitectos utilizan archivos **VEX (Vulnerability Exploitability eXchange)** para "silenciar" vulnerabilidades que, aunque existen en la librería, no pueden ser explotadas por la forma en que está configurada la aplicación, ahorrando tiempo y alertas falsas.

7. Persistencia y Redes (Escaneo de Registries)

No basta con escanear en local. Un registro privado de élite (como Harbor o AWS ECR) tiene escaneo automático. Cada vez que haces un push de una imagen, el registro la bloquea y no permite que nadie la descargue si tiene vulnerabilidades críticas.

8. Troubleshooting Forense

- **Escenario:** "Docker Scout no detecta vulnerabilidades en mi binario de Go/Rust".
- **Diagnóstico:** Los lenguajes que compilan a binarios estáticos no tienen un gestor de paquetes que Scout pueda leer fácilmente dentro de la imagen.
- **Solución de Arquitecto:** Debes generar un archivo **SBOM** durante el build y pasárselo a Scout para que sepa qué dependencias incluiste en el binario.

9. Anti-Patrones (Errores de Producción)

- **Ignorar los "Fixables":** Muchas vulnerabilidades son informativas, pero las que tienen el tag fixable deben corregirse inmediatamente, ya que existe un parche disponible.
- **Confiar en imágenes de terceros no oficiales:** Usar bitnami/nginx es más seguro que usar user123/nginx, ya que Bitnami tiene procesos de parcheo automatizados.

10. Escalabilidad y Orquestación

En Kubernetes, usamos **Admission Controllers**. Son "porteros" que preguntan: "¿Esta imagen ha pasado el escaneo de seguridad?". Si la respuesta es no, el clúster se niega a crear el Pod, garantizando que nunca entre código vulnerable en producción de forma automática.